

BANK DATABASE - TRIGGERS AND STORED PROCEDURES

STEP 1:- CREATE DATABASE

```
Database changed
[mysql> CREATE DATABASE BankDB;
Query OK, 1 row affected (0.004 sec)

[mysql> USE BankDB;
Database changed
mysql> █
```

STEP 2:- CREATE CUSTOMER TABLE

```
[mysql> USE BankDB;
Database changed
mysql> CREATE TABLE Customer (
    ->     Customer_ID INT PRIMARY KEY,
    ->     Customer_Name VARCHAR(100) NOT NULL,
    ->     Phone VARCHAR(15),
    ->     Email VARCHAR(100),
    ->     City VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.011 sec)
```

STEP 3:- CREATE ACCOUNT TABLE

```
mysql> CREATE TABLE Account (
    ->     Account_No INT PRIMARY KEY,
    ->     Customer_ID INT,
    ->     Account_Type VARCHAR(20),
    ->     Balance DECIMAL(12,2) DEFAULT 0,
    ->     Branch VARCHAR(50),
    ->
    ->     FOREIGN KEY (Customer_ID)
    ->     REFERENCES Customer(Customer_ID)
    -> );
Query OK, 0 rows affected (0.012 sec)
```

STEP 4:- CREATE BANK_TRANSACTION TABLE

```
mysql> CREATE TABLE Bank_Transaction (  
  -> Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,  
  -> Account_No INT,  
  -> Transaction_Type VARCHAR(20),  
  -> Amount DECIMAL(12,2),  
  -> Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,  
  ->  
  -> FOREIGN KEY (Account_No)  
  -> REFERENCES Account(Account_No)  
  -> );  
Query OK, 0 rows affected (0.010 sec)
```

STEP 5:- CREATE LOAN TABLE

```
mysql> CREATE TABLE Loan (  
  -> Loan_ID INT PRIMARY KEY,  
  -> Customer_ID INT,  
  -> Loan_Type VARCHAR(30),  
  -> Loan_Amount DECIMAL(12,2),  
  -> Interest_Rate DECIMAL(5,2),  
  ->  
  -> FOREIGN KEY (Customer_ID)  
  -> REFERENCES Customer(Customer_ID)  
  -> );  
Query OK, 0 rows affected (0.012 sec)
```

STEP 6:- INSERT CUSTOMER DATA

```
mysql> INSERT INTO Customer  
  -> (Customer_ID, Customer_Name, Phone, Email, City)  
  -> VALUES  
  -> (101, 'Ravi Kumar', '9876543210', 'ravi@gmail.com', 'Hyderabad'),  
  -> (102, 'Priya Sharma', '9876543211', 'priya@gmail.com', 'Vijayawada'),  
  -> (103, 'Arjun Reddy', '9876543212', 'arjun@gmail.com', 'Bangalore'),  
  -> (104, 'Sneha Rao', '9876543213', 'sneha@gmail.com', 'Chennai'),  
  -> (105, 'Kiran Kumar', '9876543214', 'kiran@gmail.com', 'Hyderabad');  
Query OK, 5 rows affected (0.003 sec)  
Records: 5 Duplicates: 0 Warnings: 0
```

STEP 7:- INSERT ACCOUNT DATA

```
mysql> INSERT INTO Account
-> (Account_No, Customer_ID, Account_Type, Balance, Branch)
-> VALUES
-> (10001, 101, 'Savings', 50000, 'Hyderabad'),
-> (10002, 102, 'Savings', 75000, 'Vijayawada'),
-> (10003, 103, 'Current', 120000, 'Bangalore'),
-> (10004, 104, 'Savings', 45000, 'Chennai'),
-> (10005, 105, 'Current', 90000, 'Hyderabad');
Query OK, 5 rows affected (0.003 sec)
Records: 5 Duplicates: 0 Warnings: 0
```

STEP 8:- INSERT TRANSACTION DATA

```
mysql> INSERT INTO Bank_Transaction
-> (Account_No, Transaction_Type, Amount)
-> VALUES
-> (10001, 'DEPOSIT', 10000),
-> (10002, 'DEPOSIT', 15000),
-> (10003, 'WITHDRAW', 20000),
-> (10004, 'DEPOSIT', 5000),
-> (10005, 'WITHDRAW', 10000);
Query OK, 5 rows affected (0.004 sec)
Records: 5 Duplicates: 0 Warnings: 0
```

STEP 9:- INSERT LOAN DATA

```
mysql> INSERT INTO Loan
-> (Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate)
-> VALUES
-> (501, 101, 'Home Loan', 5000000, 7.5),
-> (502, 102, 'Education Loan', 1000000, 6.5),
-> (503, 103, 'Car Loan', 800000, 8.2),
-> (504, 104, 'Personal Loan', 500000, 10.5);
Query OK, 4 rows affected (0.004 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

STEP 10:- DISPLAYING TABLE DATA

```
[mysql> SELECT * FROM Customer;
```

Customer_ID	Customer_Name	Phone	Email	City
101	Ravi Kumar	9876543210	ravi@gmail.com	Hyderabad
102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad

```
5 rows in set (0.000 sec)
```

```
[mysql> SELECT * FROM Account;
```

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad
10002	102	Savings	75000.00	Vijayawada
10003	103	Current	120000.00	Bangalore
10004	104	Savings	45000.00	Chennai
10005	105	Current	90000.00	Hyderabad

```
5 rows in set (0.000 sec)
```

```
[mysql> SELECT * FROM Bank_Transaction;
```

Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
1	10001	DEPOSIT	10000.00	2026-08-18 14:04:27
2	10002	DEPOSIT	15000.00	2026-08-18 14:04:27
3	10003	WITHDRAW	20000.00	2026-08-18 14:04:27
4	10004	DEPOSIT	5000.00	2026-08-18 14:04:27
5	10005	WITHDRAW	10000.00	2026-08-18 14:04:27

```
5 rows in set (0.000 sec)
```

```
mysql> SELECT * FROM Loan;
```

Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate
501	101	Home Loan	5000000.00	7.50
502	102	Education Loan	1000000.00	6.50
503	103	Car Loan	800000.00	8.20
504	104	Personal Loan	500000.00	10.50

```
4 rows in set (0.000 sec)
```


STORED PROCEDURES

STEP 11:- PROCEDURE TO DISPLAY ALL CUSTOMERS

```
[mysql> DELIMITER //
mysql> CREATE PROCEDURE GetAllCustomers()
  -> BEGIN
  ->     SELECT * FROM Customer;
  -> END //
Query OK, 0 rows affected (0.011 sec)

[mysql> DELIMITER ;
mysql> CALL GetAllCustomers();
```

Customer_ID	Customer_Name	Phone	Email	City
101	Ravi Kumar	9876543210	ravi@gmail.com	Hyderabad
102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad

```
5 rows in set (0.002 sec)

Query OK, 0 rows affected (0.002 sec)
```

STEP 12:- PROCEDURE TO DISPLAY ACCOUNT DETAILS

```
[mysql> DELIMITER ;
mysql> CALL GetAccountDetails(10001);
```

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad

```
1 row in set (0.001 sec)

Query OK, 0 rows affected (0.001 sec)
```

STEP 13:- PROCEDURE TO FIND CUSTOMER ACCOUNTS

```
[mysql> DELIMITER ;
mysql> CALL GetCustomerAccounts(101);
```

Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Branch
101	Ravi Kumar	10001	Savings	50000.00	Hyderabad

```
1 row in set (0.001 sec)

Query OK, 0 rows affected (0.001 sec)
```

STEP 14:- PROCEDURE TO DEPOSIT MONEY

```
[mysql> CALL DepositMoney(10001, 5000);
Query OK, 1 row affected (0.003 sec)

mysql> SELECT * FROM Account
[    -> WHERE Account_No = 10001;
+-----+-----+-----+-----+-----+
| Account_No | Customer_ID | Account_Type | Balance | Branch |
+-----+-----+-----+-----+-----+
|      10001 |          101 | Savings      | 55000.00 | Hyderabad |
+-----+-----+-----+-----+-----+
1 row in set (0.001 sec)
```

STEP 15:- PROCEDURE TO WITHDRAW MONEY

```
[mysql> CALL WithdrawMoney(10001, 3000);
Query OK, 1 row affected (0.003 sec)

mysql> SELECT * FROM Account
[    -> WHERE Account_No = 10001;
+-----+-----+-----+-----+-----+
| Account_No | Customer_ID | Account_Type | Balance | Branch |
+-----+-----+-----+-----+-----+
|      10001 |          101 | Savings      | 52000.00 | Hyderabad |
+-----+-----+-----+-----+-----+
1 row in set (0.001 sec)
```

TRIGGERS

STEP 16:- TRIGGER TO PREVENT NEGATIVE BALANCE

```
[mysql> DELIMITER ;
mysql> UPDATE Account
    -> SET Balance = Balance - 60000
[    -> WHERE Account_No = 10001;
ERROR 1644 (45000): Transaction failed: Insufficient balance
mysql> SELECT * FROM Account
[    -> WHERE Account_No = 10001;
+-----+-----+-----+-----+-----+
| Account_No | Customer_ID | Account_Type | Balance | Branch |
+-----+-----+-----+-----+-----+
|      10001 |          101 | Savings      | 52000.00 | Hyderabad |
+-----+-----+-----+-----+-----+
1 row in set (0.000 sec)
```

STEP 17:- TRIGGER TO PREVENT NEGATIVE TRANSACTION AMOUNT

```
[mysql> DELIMITER //
mysql> CREATE TRIGGER CheckTransactionAmount
  -> BEFORE INSERT ON Bank_Transaction
  -> FOR EACH ROW
  -> BEGIN
  ->     IF NEW.Amount <= 0 THEN
  ->         SIGNAL SQLSTATE '45000'
  ->         SET MESSAGE_TEXT =
  ->         'Transaction amount must be greater than zero';
  ->     END IF;
  -> END //
Query OK, 0 rows affected (0.006 sec)

[mysql> DELIMITER ;
mysql> INSERT INTO Bank_Transaction
  -> (Account_No, Transaction_Type, Amount)
  -> VALUES
  -> (10001, 'DEPOSIT', -5000);
ERROR 1644 (45000): Transaction amount must be greater than zero
```

STEP 18:- CREATE TRANSACTION AUDIT TABLE

```
mysql> CREATE TABLE Transaction_Audit (
  ->     Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
  ->     Transaction_ID INT,
  ->     Account_No INT,
  ->     Transaction_Type VARCHAR(20),
  ->     Amount DECIMAL(12,2),
  ->     Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
  -> );
Query OK, 0 rows affected (0.011 sec)
```

```
mysql> DESCRIBE Transaction_Audit;
```

Field	Type	Null	Key	Default	Extra
Audit_ID	int	NO	PRI	NULL	auto_increment
Transaction_ID	int	YES		NULL	
Account_No	int	YES		NULL	
Transaction_Type	varchar(20)	YES		NULL	
Amount	decimal(12,2)	YES		NULL	
Audit_Date	datetime	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

```
6 rows in set (0.006 sec)
```

STEP 19:- TRIGGER FOR TRANSACTION AUDIT

```
[mysql> SELECT * FROM Transaction_Audit;
```

Audit_ID	Transaction_ID	Account_No	Transaction_Type	Amount	Audit_Date
1	6	10001	DEPOSIT	2500.00	2026-08-18 14:38:13

```
1 row in set (0.000 sec)
```

STEP 20:- TRIGGER TO AUTOMATICALLY UPDATE ACCOUNT BALANCE

```
mysql> INSERT INTO Bank_Transaction
      -> (Account_No, Transaction_Type, Amount)
      -> VALUES
[      -> (10001, 'DEPOSIT', 5000);
Query OK, 1 row affected (0.004 sec)
```

```
mysql> SELECT *
      -> FROM Account
[      -> WHERE Account_No = 10001;
```

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	57000.00	Hyderabad

1 row in set (0.000 sec)

STEP 21:- TRIGGER TO PREVENT WITHDRAWAL ABOVE BALANCE

```
[mysql> DELIMITER ;
mysql> INSERT INTO Bank_Transaction
      -> (Account_No, Transaction_Type, Amount)
      -> VALUES
[      -> (10001, 'WITHDRAW', 1000000);
ERROR 1644 (45000): Withdrawal failed: Insufficient balance
mysql> SELECT *
      -> FROM Account
[      -> WHERE Account_No = 10001;
```

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	57000.00	Hyderabad

1 row in set (0.000 sec)

ADVANCED STORED PROCEDURES

STEP 22:- PROCEDURE FOR ACCOUNT TRANSFER

```
[mysql> CALL TransferMoney(10001, 10002, 5000);
Query OK, 1 row affected (0.002 sec)

mysql> SELECT *
      -> FROM Account
      -> WHERE Account_No IN (10001,10002);
```

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	52000.00	Hyderabad
10002	102	Savings	80000.00	Vijayawada

2 rows in set (0.000 sec)

STEP 23:- PROCEDURE FOR CUSTOMER LOAN DETAILS

```
[mysql> DELIMITER ;
[mysql> CALL GetCustomerLoans(101);
```

Customer_Name	Loan_ID	Loan_Type	Loan_Amount	Interest_Rate
Ravi Kumar	501	Home Loan	5000000.00	7.50

1 row in set (0.001 sec)

Query OK, 0 rows affected (0.001 sec)

STEP 24 — PROCEDURE FOR HIGH BALANCE ACCOUNTS

```
[mysql> DELIMITER ;
[mysql> CALL HighBalanceAccounts(50000);
```

Account_No	Customer_ID	Account_Type	Balance	Branch
10003	103	Current	120000.00	Bangalore
10005	105	Current	90000.00	Hyderabad
10002	102	Savings	80000.00	Vijayawada
10001	101	Savings	52000.00	Hyderabad

4 rows in set (0.001 sec)

Query OK, 0 rows affected (0.001 sec)

STEP 25 — PROCEDURE WITH IN & OUT PARAMETERS

```
[mysql> DELIMITER ;  
[mysql> CALL GetBalance(10001, @CurrentBalance);  
Query OK, 1 row affected (0.003 sec)  
  
[mysql> SELECT @CurrentBalance;  
+-----+  
| @CurrentBalance |  
+-----+  
|          52000.00 |  
+-----+  
1 row in set (0.000 sec)
```